

Django Box Selection Roadmap

Exported with **GPTAha Free**

[Remove watermark & unlock unlimited PDFs →](#)

Python/Django Hiring Assignment: AI-Assisted Box Selection System

Context

We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost.

Your task is to design and build a small Django-based system that recommends the most suitable box for an order.

You are allowed to use AI tools such as ChatGPT, Claude, Cursor, GitHub Copilot, or similar tools. However, your submission must clearly show your own thinking.

AI Usage

You may use AI tools, but include an `AI_USAGE.md` file containing:

1. Which AI tool(s) you used.
2. The prompts you gave.
3. What output you accepted.
4. What output you rejected or modified.
5. Any mistakes the AI made.
6. How you verified the final code.

Submission

Please submit:

1. GitHub repository link.
2. `README.md`.
3. `AI_USAGE.md` describing tools, prompts, accepted/rejected outputs, mistakes found, and verification steps.
4. Export the chat transcript and attach and include in your repository. Make sure that you don't generate this using AI. Your assignment will be rejected immediately if we see that you used AI for this.

5. What did you learn in this assignment? Again, not to use AI to generate an answer for this.
6. Test cases.
7. Test run output, either:
 - GitHub Actions link,
 - terminal output pasted in TEST_OUTPUT.md. analyze this and create a proper map to build this analyze this properly and give me proper roadmap

Yes. This assignment is actually a **small but very good Django backend project**. The important part is not just making CRUD screens—it is designing a **correct box-selection algorithm**, exposing it through Django, testing edge cases, and documenting where AI helped versus where *you* made the decisions.

I would build it in the following order.

1. Understand exactly what you are being asked to build

The system has four main concepts:

PRODUCT

- └─ name
- └─ length
- └─ width
- └─ height
- └─ weight

BOX

- └─ name
- └─ internal_length
- └─ internal_width
- └─ internal_height
- └─ max_weight
- └─ cost

ORDER

- └─ contains multiple products + quantities

BOX RECOMMENDER

- └─ receives an order
- └─ calculates total dimensions/weight
- └─ checks which boxes can contain it
- └─ recommends the most suitable box

The core flow should be:

Customer Order

↓

Order Items

↓

Calculate total product weight

↓

Determine whether products fit inside each box

↓

Filter boxes that satisfy:

- dimensions
- weight capacity

↓

Rank valid boxes

↓

Return recommended box

2. Decide the business rules BEFORE coding

This is one of the most important parts of the assignment.

The assignment doesn't completely specify the packing algorithm. You therefore need to make your assumptions explicit in `README.md`.

I recommend keeping the first version simple and defensible.

Recommended assumptions

A. Product rotation is allowed

For example:

```
Product:
10 × 5 × 4
```

```
Box:
6 × 5 × 10
```

The product fits because you can rotate it.

Therefore:

```
product dimensions
→ try all 6 orientations
→ if one orientation fits → product fits
```

B. Weight is additive

If an order contains:

```
Product A = 2 kg
quantity = 2
```

```
Product B = 1 kg
quantity = 1
```

Then:

```
total weight = 2 × 2 + 1 × 1
              = 5 kg
```

The box must satisfy:

```
box.max_weight >= total_order_weight
```

C. Multiple products must physically fit

This is the difficult part.

Don't simply do:

```
max(product.length) <= box.length  
max(product.width) <= box.width  
max(product.height) <= box.height
```

That does **not** prove that multiple products can physically be packed into the box.

For a hiring assignment, you have two reasonable approaches:

Version 1 — simple packing

Implement a constrained packing algorithm suitable for the assignment and clearly document its limitations.

Version 2 — more sophisticated 3D packing

Implement a proper 3D bin-packing approach.

I recommend Version 1 initially.

The interviewer will care more about:

- clean architecture
- correctness
- tests
- assumptions
- maintainability
- explanation

than whether you built an industrial-grade warehouse optimization engine.

3. Define the recommendation rule

You need to answer:

| If several boxes can hold an order, which box should we choose?

A sensible rule is:

1. Box must fit all products.

2. Box must support total weight.

3. Among valid boxes:

choose the box with the lowest cost.

4. If cost is equal:

choose the smallest-volume box.

For example:

Box	Fits?	Weight OK?	Cost	Volume
Small	Yes	Yes	₹50	1,000
Medium	Yes	Yes	₹80	2,000
Large	Yes	Yes	₹120	5,000

Recommendation:

Small

This is easy to explain and test.

You can also state:

"The recommendation prioritizes minimum shipping-box cost.
Volume is used only as a deterministic tie-breaker."

That's a strong design decision.

4. Recommended project architecture

Don't put everything inside `views.py` .

Use something like:

```
box-selection/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── inventory/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── serializers.py
│   ├── urls.py
│   ├── views.py
│   └── tests/
│       ├── __init__.py
│       ├── test_models.py
│       ├── test_packing.py
│       └── test_api.py
├── recommender/
│   ├── __init__.py
│   ├── packing.py
│   └── service.py
├── requirements.txt
├── README.md
├── AI_USAGE.md
├── TEST_OUTPUT.md
├── .gitignore
└── .github/
    └── workflows/
        └── tests.yml
```

This is much better than making one giant Django app.

5. Database design

I would use **three primary models**.

Product

```
Product
-----
id
name
length
width
height
weight
created_at
```

Use `DecimalField` for dimensions and weight rather than floating-point fields.

For example:

```
length = models.DecimalField(max_digits=10, decimal_places=2)
```

Box

```
Box
---
id
name
internal_length
internal_width
internal_height
max_weight
cost
created_at
```

Again, use `DecimalField`.

Order

```
Order
-----
id
created_at
```

OrderItem

```
OrderItem
-----
id
order
product
quantity
```

Relationship:

```
Order
|
|— OrderItem → Product
|
|— OrderItem → Product
|
|— OrderItem → Product
```

This gives you:

```
Order #1

2 × Laptop
1 × Mouse
3 × Keyboard
```

without duplicating product information.

6. API design

Keep the API small.

You don't need 20 endpoints.

I would build:

Products

```
POST /api/products/
GET  /api/products/
GET  /api/products/{id}/
```

Boxes

```
POST /api/boxes/  
GET /api/boxes/  
GET /api/boxes/{id}/
```

Orders

```
POST /api/orders/  
GET /api/orders/{id}/
```

Recommendation

Most important endpoint:

```
POST /api/orders/{id}/recommend-box/
```

Response:

```
{  
  "order_id": 12,  
  "recommended_box": {  
    "id": 3,  
    "name": "Medium Box",  
    "cost": "80.00"  
  },  
  "total_weight": "4.50",  
  "candidate_boxes": [  
    {  
      "id": 3,  
      "name": "Medium Box",  
      "cost": "80.00"  
    },  
    {  
      "id": 4,  
      "name": "Large Box",  
      "cost": "120.00"  
    }  
  ]  
}
```

If nothing fits:

```
{  
  "order_id": 12,  
  "recommended_box": null,  
  "reason": "No available box can contain the order."  
}
```

That's much better than returning a generic HTTP 500.

7. Separate the algorithm from Django

This is probably the biggest architectural recommendation I'd give you.

Don't do this:

```
def recommend_box(request):  
    # 150 lines of packing logic
```

Instead:

```
Django API  
  ↓  
Recommendation Service  
  ↓  
Packing Algorithm
```

For example:

```
recommend_box(order, boxes)
```

and:

```
can_fit_product(product, box)
```

and potentially:

```
can_pack_order(items, box)
```

This allows you to test your core algorithm without even starting Django.

That demonstrates good engineering thinking.

8. Packing algorithm roadmap

Start with the smallest useful algorithm.

Step 1 — weight check

```
total_weight = sum(  
    item.product.weight * item.quantity  
    for item in order.items.all()  
)
```

Reject box if:

```
total_weight > box.max_weight
```

Step 2 — dimension check

For every product:

```
try:  
  
    L × W × H  
  
    L × H × W  
  
    W × L × H  
  
    W × H × L  
  
    H × L × W  
  
    H × W × L
```

If any orientation satisfies:

```
product_L <= box_L  
product_W <= box_W  
product_H <= box_H
```

then the individual product fits.

9. Handling multiple products

This is where you need to make your design choice.

A simple first implementation could use a **remaining-space / placement strategy**.

Conceptually:

BOX

```
+-----+
| Product A      |
|               |
|-----+-----|
| Product B    | Product C |
|               |           |
+-----+
```

For each product:

1. Try each orientation.
2. Try available placement positions.
3. If a valid placement exists:
 place product
4. Otherwise:
 box doesn't fit

You can model free spaces.

For example:

```
FreeSpace(  
    x,  
    y,  
    z,  
    length,  
    width,  
    height  
)
```

Initially:

```
FreeSpace(  
    0,  
    0,  
    0,  
    box.length,  
    box.width,  
    box.height  
)
```

When you place an object, split the remaining space.

This is considerably more interesting technically and gives you something good to discuss in the interview.

10. But don't over-engineer immediately

Your development phases should be:

```
PHASE 1  
Basic Django models  
    ↓  
PHASE 2  
CRUD/API  
    ↓  
PHASE 3  
Simple box eligibility  
    ↓  
PHASE 4  
Recommendation logic  
    ↓  
PHASE 5  
Multi-product packing  
    ↓  
PHASE 6  
Testing  
    ↓  
PHASE 7  
Documentation  
    ↓  
PHASE 8  
CI/CD
```

Don't start by trying to write a complicated 3D packing engine.

11. Test cases you absolutely need

This assignment explicitly asks for test cases, so this should be a major part of the project.

I'd create at least these.

Test 1 — product fits normally

Product: $10 \times 10 \times 10$
Box: $20 \times 20 \times 20$

Expected:

PASS

Test 2 — product fits after rotation

Product: $10 \times 5 \times 4$
Box: $6 \times 5 \times 10$

Expected:

PASS

Test 3 — dimensions don't fit

Product: $30 \times 20 \times 10$
Box: $20 \times 20 \times 20$

Expected:

FAIL

Test 4 — weight exceeds capacity

Product weight = 10 kg
Box capacity = 5 kg

Expected:

FAIL

Test 5 — exact dimension match

Product: 10 × 10 × 10
Box: 10 × 10 × 10

Expected:

PASS

Important boundary test.

Test 6 — multiple quantities

Product weight = 2 kg
Quantity = 3

Total = 6 kg

Verify weight calculation.

Test 7 — cheapest valid box

Box A = ₹100
Box B = ₹70
Box C = ₹150

If all fit:

recommended = Box B

Test 8 — cheapest box doesn't fit

```
Small = ₹50 → doesn't fit  
Medium = ₹80 → fits  
Large = ₹100 → fits
```

Expected:

```
Medium
```

Test 9 — no box available

Expected:

```
recommended_box = None
```

Test 10 — empty order

Expected:

```
validation error
```

Test 11 — invalid quantity

```
quantity = 0  
quantity = -1
```

Should be rejected.

Test 12 — multiple products

Example:

```
2 × Product A  
1 × Product B  
3 × Product C
```

Verify:

```
total weight
packing
recommended box
```

12. API test example

You should test the actual endpoint too.

For example:

```
POST /api/orders/1/recommend-box/
```

Then assert:

```
response.status_code == 200
```

and:

```
response.data["recommended_box"]["name"] == "Medium Box"
```

This shows the interviewer that you're testing the **whole system**, not only individual functions.

13. Admin interface

Add Django Admin.

This makes your project immediately easier to demonstrate.

You should be able to open:

```
/admin/
```

and create:

Products
Boxes
Orders
Order Items

This is a small addition with a lot of demo value.

14. README structure

Your README should look approximately like:

```
# AI-Assisted Box Selection System

## Overview

## Problem Statement

## Features

## Technology Stack

## Architecture

## Database Design

## Box Selection Algorithm

## Business Rules / Assumptions

## API Endpoints

## Example Request

## Example Response

## Installation

## Running Locally

## Running Tests

## Test Coverage

## AI Usage

## Limitations

## Future Improvements
```

Especially document **assumptions**.

That's where you show your own thinking.

15. AI_USAGE.md is extremely important

Don't try to hide AI usage.

They explicitly allow it.

Instead, make it transparent.

Example structure:

AI Usage

Tools Used

- ChatGPT
- GitHub Copilot

Why I Used AI

I used AI mainly for:

- understanding Django patterns
- reviewing code structure
- generating initial test ideas
- debugging specific errors

Prompts Used

Prompt 1

[Your actual prompt]

Prompt 2

[Your actual prompt]

Output Accepted

[Explain what you actually used]

Output Modified

[Explain what you changed]

Output Rejected

[Explain what you didn't use and why]

Mistakes Found

1. ...
2. ...
3. ...

Verification

I verified the generated code by:

- running Django tests
- testing API endpoints
- checking edge cases
- manually testing the recommendation logic

Important: use your actual prompts and actual experience. Don't fabricate an AI history.

16. The chat transcript requirement

This is the one part you should treat differently.

They specifically say:

"Make sure that you don't generate this using AI."

So **do not ask me to create that transcript.**

Keep the actual ChatGPT/Copilot/etc. conversation export generated through the relevant platform's normal export mechanism.

Do not rewrite it to make it look better.

Put the resulting file in something like:

```
docs/  
└─ AI_CHAT_TRANSCRIPT.pdf
```

or whatever format the hiring company accepts.

17. "What did you learn?" requirement

Same principle.

Don't have AI write this section for you.

The company is explicitly checking whether you can reflect on your own work.

Instead, after finishing the project, write down your own answers to questions like:

What was the hardest part?

What did I initially misunderstand?

What did I change after testing?

What Django concept did I learn?

What did I learn about designing algorithms?

What did AI suggest that turned out to be wrong?

How did I verify AI-generated code?

What would I improve if I had another week?

Then put your own response into the repository.

18. TEST_OUTPUT.md

Run:

```
python manage.py test
```

You want something along the lines of:

```
Creating test database...
```

```
.....
```

```
-----
```

```
Ran 12 tests in 0.XXXs
```

```
OK
```

```
Destroying test database...
```

Paste your **actual terminal output** into:

```
TEST_OUTPUT.md
```

Don't manually manufacture a successful output.

19. GitHub Actions

I strongly recommend adding this.

Repository:

```
.github/  
└─ workflows/  
   └─ tests.yml
```

Pipeline:

```
Push code  
  ↓  
GitHub Actions  
  ↓  
Install Python  
  ↓  
Install dependencies  
  ↓  
Run migrations  
  ↓  
Run tests  
  ↓  
PASS / FAIL
```

Then your README can show:

```
Tests: passing
```

with the actual GitHub Actions link/badge.

20. Development roadmap

Here's the roadmap I'd personally follow.

Day 1 — Project foundation

- ❑ Create GitHub repository
- ❑ Create virtual environment
- ❑ Install Django
- ❑ Install Django REST Framework
- ❑ Create Django project
- ❑ Create inventory app
- ❑ Configure SQLite
- ❑ Create .gitignore
- ❑ Initial commit

Day 2 — Database

- ❑ Product model
- ❑ Box model
- ❑ Order model
- ❑ OrderItem model
- ❑ Migrations
- ❑ Django admin
- ❑ Model validation
- ❑ Seed/sample data

Day 3 — API

- ❑ Product API
- ❑ Box API
- ❑ Order API
- ❑ Serializer validation
- ❑ API error handling
- ❑ API documentation/examples

Day 4 — Recommendation engine

- ❑ Calculate order weight
- ❑ Product orientation logic
- ❑ Dimension validation
- ❑ Box eligibility
- ❑ Multi-product packing
- ❑ Box ranking
- ❑ Recommendation service

This is your **core technical day**.

Day 5 — Tests

- ☐ Unit tests
- ☐ Packing tests
- ☐ Model tests
- ☐ API tests
- ☐ Edge cases
- ☐ No-box scenario
- ☐ Invalid input tests
- ☐ Run complete suite

Day 6 — Documentation

- ☐ README
- ☐ Architecture explanation
- ☐ Algorithm explanation
- ☐ Assumptions
- ☐ API examples
- ☐ AI_USAGE.md
- ☐ TEST_OUTPUT.md
- ☐ Your own learning notes
- ☐ Actual chat transcript

Day 7 — Polish

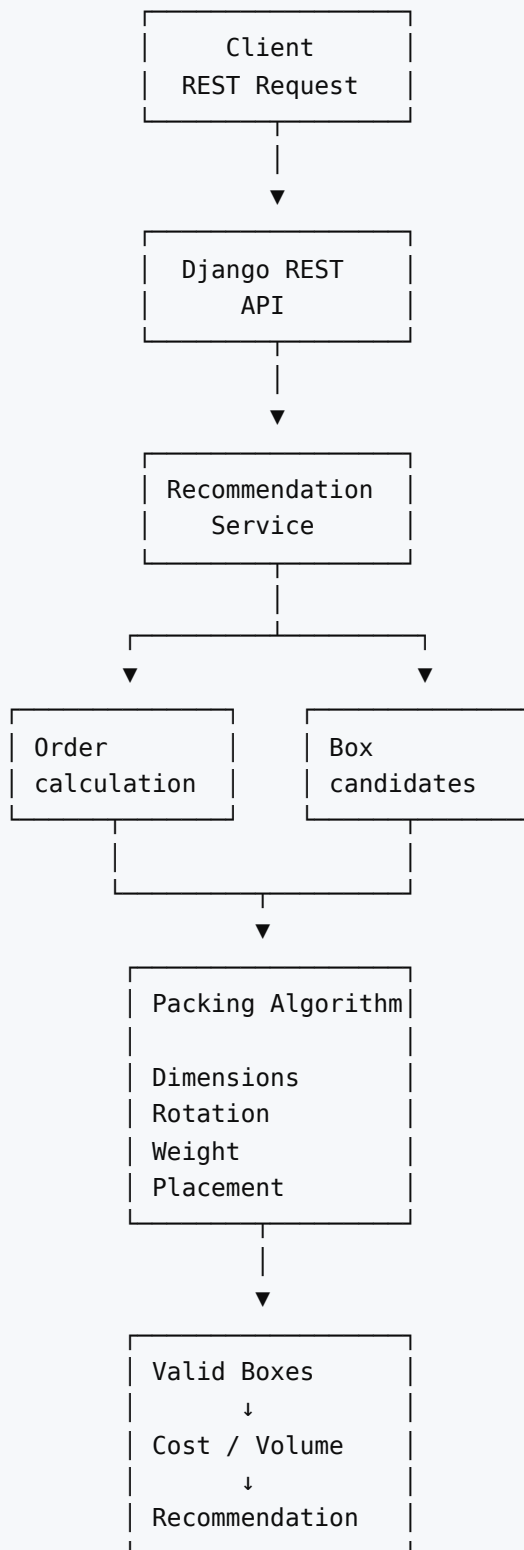
- ☐ GitHub Actions
- ☐ Clean code
- ☐ Remove unused files
- ☐ Check secrets
- ☐ Check requirements.txt
- ☐ Test from fresh clone
- ☐ Review README
- ☐ Review AI disclosure
- ☐ Final GitHub push

21. Final repository should look like this

```
box-selection-system/
├── .github/
│   └── workflows/
│       └── tests.yml
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── ...
├── inventory/
│   ├── migrations/
│   ├── tests/
│   │   ├── test_models.py
│   │   ├── test_packing.py
│   │   └── test_api.py
│   ├── admin.py
│   ├── models.py
│   ├── serializers.py
│   ├── urls.py
│   └── views.py
├── recommender/
│   ├── packing.py
│   └── service.py
├── docs/
│   └── AI_CHAT_TRANSCRIPT.pdf
├── manage.py
├── requirements.txt
├── README.md
├── AI_USAGE.md
├── TEST_OUTPUT.md
├── .gitignore
└── LICENSE
```

22. The architecture you should be able to explain in the interview

The whole system can be explained in one diagram:



23. What will make this submission stand out

Don't try to make it huge.

Focus on these **six things**:

1. Clean separation

```
API ≠ business logic ≠ packing algorithm
```

2. Explicit assumptions

Tell them exactly what your algorithm considers "fits."

3. Good edge-case testing

Especially:

```
rotation
exact dimensions
weight limit
multiple products
quantity
no suitable box
invalid input
```

4. Explainable recommendation

Don't just return:

```
{"box": 3}
```

Explain why:

```
{
  "recommended_box": "Medium"
```

ohk i understand what we have to build as they are saying they want AI-Assisted Box Selection System here how we are implementing ai assistance ? explain me in short and effective maner

The key point is: **AI assistance does not necessarily mean the box-selection algorithm itself must use an LLM.**

For this assignment, I would implement AI as a **supporting decision/explanation layer**, while keeping the actual box-fit

calculation deterministic.

Simple architecture

```
graph TD;
    Order --> DjangoBackend[Django Backend];
    DjangoBackend --> BoxSelection[Box Selection Algorithm];
    BoxSelection --> FindValidBoxes[Find valid boxes];
    FindValidBoxes --> ChooseSuitableBox[Choose suitable box];
    ChooseSuitableBox --> AIExplanationLayer[AI Explanation Layer];
    AIExplanationLayer --> WhySelected["Why this box was selected?"];
```

What AI can do

Suppose the order contains:

Laptop: 30×20×3 cm, 2 kg
Mouse: 10×6×4 cm, 0.2 kg

The normal algorithm determines:

Recommended Box: Medium Box
Cost: ₹80
Weight: 2.2 kg

Then AI can generate a human-friendly explanation:

Medium Box is recommended because it can accommodate all items within its internal dimensions and supports the total weight of 2.2 kg. It is also cheaper than the other boxes that satisfy these requirements.

So:

Algorithm = makes the decision

AI = explains/assists with the decision

Why I recommend this approach

It demonstrates good engineering judgment:

- You **don't rely on an LLM for mathematical/dimensional validation**.
- Results are deterministic and testable.
- AI adds useful functionality rather than being artificially inserted.
- You can explain exactly where AI is used during the interview.
- If the AI service fails, the core box recommendation still works.

Optional AI feature

You could also let a warehouse employee ask:

"Why wasn't the small box selected?"

AI could respond:

"The small box was rejected because its maximum weight capacity is 2 kg, while the order weighs 2.2 kg."

But the **actual reason should come from your backend's calculated data**, not from AI guessing.

So your project positioning becomes:

"We use a deterministic packing and cost-optimization engine for the actual box selection, and an AI layer to provide natural-language explanations and assist warehouse users in understanding the recommendation."

That's a much stronger implementation than simply calling ChatGPT and asking *"Which box should I use?"*

What if we also include evaluation and rejection reasons ? so just spitting out one recommendation, the system could break down why others failed like dimensions not fitting n all. user can get proper analysis and can select proper box for product

Yes — **that's actually a much better design** for this assignment.

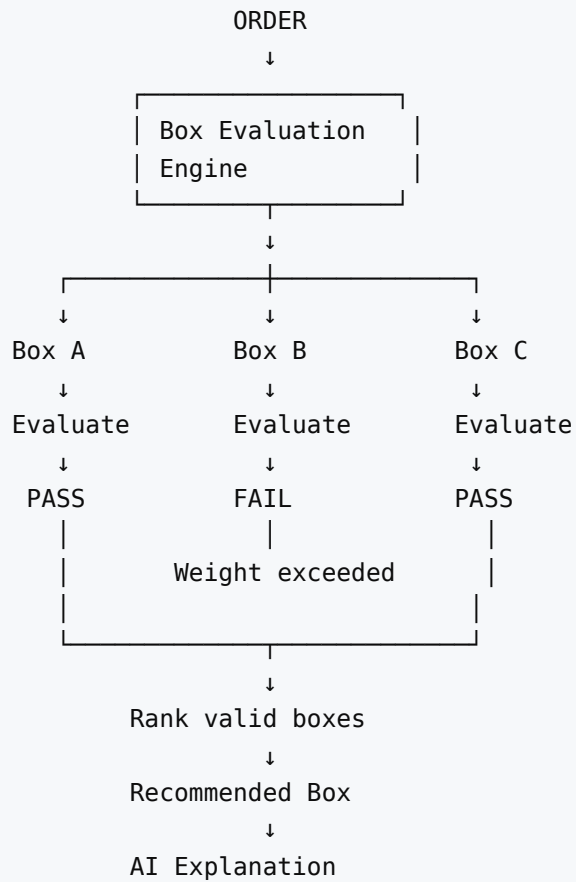
Instead of making the system:

Input order → AI → Box recommendation

make it:

Input order → Deterministic evaluation → Evaluate EVERY box → Recommendation + rejection analysis → AI explanation

Recommended flow



Example

Suppose there are 4 boxes.

The API could return:

```
{
  "recommended_box": {
    "name": "Medium Box",
    "cost": 80
  },
  "evaluation": [
    {
      "box": "Small Box",
      "status": "rejected",
      "reasons": [
        "Product dimensions exceed box height",
        "Maximum weight capacity is 2 kg, order weight is 3.5 kg"
      ]
    },
    {
      "box": "Medium Box",
      "status": "valid",
      "reasons": []
    },
    {
      "box": "Large Box",
      "status": "valid",
      "reasons": []
    },
    {
      "box": "XL Box",
      "status": "valid",
      "reasons": []
    }
  ]
}
```

Then your UI can show:

Recommended Box

📦 Medium Box

Cost: ₹80

- ✓ Dimensions fit
- ✓ Weight within capacity
- ✓ Lowest cost among valid boxes

Other Boxes

✗ Small Box

- └─ Dimensions don't fit
- └─ Weight capacity exceeded

✓ Large Box

- └─ Dimensions fit
- └─ Weight within capacity

Cost: ₹120

✓ XL Box

- └─ Dimensions fit
- └─ Weight within capacity

Cost: ₹180

Where AI becomes useful

This gives you a **real AI-assisted system** rather than AI being unnecessarily responsible for the core calculation.

Your backend generates structured evaluation data:

Box → PASS/FAIL

↓

Dimensions

Weight

Cost

Failure reasons

Then AI can turn that into a natural-language explanation:

"The Medium Box is recommended because all products fit within its internal dimensions and the total order weight is within its 10 kg capacity. The Small Box was rejected because the laptop exceeds its height limit and the total

weight is above its 2 kg capacity. The Large Box also fits, but costs ₹40 more."

Even better: let the user ask questions

You could have an AI assistant:

Warehouse Assistant 🤖

User:

Why can't I use the Small Box?

AI:

The Small Box was rejected for two reasons:

1. The laptop is too tall for the box.
2. The order weighs 3.5 kg, while the box supports only 2 kg.

Or:

User:

Why did you choose Medium instead of Large?

AI:

Both boxes can accommodate the order. Medium was selected because it costs ₹80 compared with ₹120 for Large.

The important technical principle

****Don't let AI**

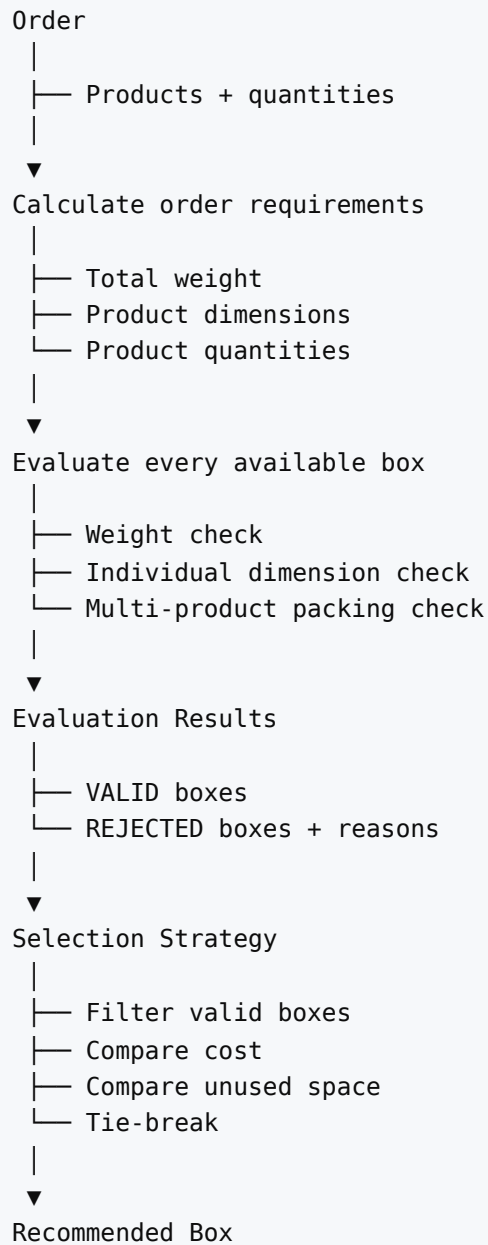
ohk can you help me design the core strategy for dimension n all also give me selection strategy

Absolutely. I'd design the core engine around **two separate responsibilities**:

1. **Evaluation engine** → determines whether every box can handle the order and gives exact rejection reasons.
2. **Selection engine** → ranks only the valid boxes and chooses the recommended one.

This makes the system deterministic, explainable, and easy to test.

1. Core strategy



2. Dimension strategy

This is the most important part.

Every product has:

Length × Width × Height

Every box has:

Internal Length × Internal Width × Internal Height

You should **allow rotation**.

For a product:

L = 30
W = 20
H = 10

there are 6 possible orientations:

30 × 20 × 10
30 × 10 × 20

20 × 30 × 10
20 × 10 × 30

10 × 30 × 20
10 × 20 × 30

For every orientation:

product_dimension ≤ box_dimension

If **at least one orientation works**, the product can potentially fit.

3. Don't just compare dimensions blindly

For example:

Product = 30 × 20 × 10

Box = 35 × 25 × 15

Obviously:

30 <= 35
20 <= 25
10 <= 15

So it fits.

But consider:

Product A = 20 × 20 × 10
Product B = 20 × 20 × 10

Box = 20 × 20 × 15

Each product individually fits.

But **both products cannot simply occupy the same physical space.**

Therefore:

Individual dimension validation is necessary, but it is not sufficient for multi-product orders.

This distinction is worth mentioning in your interview.

4. Use a two-stage dimension evaluation

I'd implement:

Stage 1 — quick rejection

For every product:

Can this product fit inside the box in ANY orientation?

If no:

REJECT

Reason:

"Product Laptop cannot fit within the internal dimensions of Small Box."

This prevents unnecessary packing calculations.

Stage 2 — actual packing

If every product passes the individual check:

Can all products physically coexist inside this box?

That's where your packing algorithm comes in.

5. A practical packing strategy

For a hiring assignment, I'd use a **3D placement algorithm based on available spaces**.

Represent an available region as:

```
FreeSpace(  
    x,  
    y,  
    z,  
    length,  
    width,  
    height  
)
```

Initially, the whole box is one free space:



When you place a product:

```

+-----+
| Product A |
|-----+
|           |
|           | remaining space
|           |
|-----+

```

you split the remaining area into new free spaces.

Conceptually:

Before:

```

+-----+
|       |
|       | FREE SPACE
|       |
|-----+

```

Place Product A:

```

+-----+-----+
| Product A |       |
|           | Free  |
+-----+-----+
|           |
|           | Free
|           |
+-----+-----+

```

Then try placing the next product in one of the available spaces.

6. Product ordering matters

Before packing, sort products from **largest to smallest**.

A simple measure is volume:

```

volume = length * width * height

```

Then:

```
Largest product
↓
Second largest
↓
Third largest
↓
...
```

Why?

Large products have fewer possible placements, so you want to place them first.

Example:

```
Laptop → 30×20×5
Monitor → 40×30×10
Mouse → 10×5×3
Keyboard → 30×10×3
```

Sort by volume:

```
Monitor
Laptop
Keyboard
Mouse
```

Then attempt placement in that order.

7. Weight strategy

Weight should be checked **before packing**.

Calculate:

```
total_weight = Σ(product.weight × quantity)
```

Then:

```
if total_weight > box.max_weight:
    reject
```

Your rejection reason should be precise:

Order weight: 8.5 kg
Box capacity: 5 kg

→ REJECTED

Not just:

"Box doesn't fit."

8. Dimension rejection reasons

Make the evaluation engine return structured reasons.

For example:

```
{
  "status": "rejected",
  "reasons": [
    {
      "type": "DIMENSION",
      "product": "Laptop",
      "message": "No orientation of the product fits inside the box."
    }
  ]
}
```

For weight:

```
{
  "type": "WEIGHT",
  "message": "Order weighs 8.5 kg but box capacity is 5 kg."
}
```

This is excellent for your AI layer later.

9. Selection strategy

Now suppose you have:

Small → rejected
Medium → valid → ₹80
Large → valid → ₹120
XL → valid → ₹150

Don't let AI decide.

Your deterministic selection engine should decide.

I recommend:

Priority 1 — Validity

Remove all boxes that fail.

VALID BOXES ONLY

Priority 2 — Cost

Among valid boxes:

lowest cost wins

So:

Medium ₹80
Large ₹120
XL ₹150

→ Medium is selected.

Priority 3 — Volume as tie-breaker

If:

Box A → ₹80 → 10,000 cm³
Box B → ₹80 → 15,000 cm³

select:

Box A

because it provides less unused space.

10. But add "space utilization"

You can make the selection strategy more intelligent.

Calculate:

```
box_volume = L × W × H
```

and:

```
total_product_volume =  
Σ(product_volume × quantity)
```

Then:

```
utilization =  
total_product_volume / box_volume
```

Example:

```
Medium:  
Product volume = 8,000  
Box volume      = 10,000  
  
Utilization = 80%
```

Large:

```
Product volume = 8,000  
Box volume      = 20,000  
  
Utilization = 40%
```

Medium is more space-efficient.

Important: volume utilization should be a supporting metric, not proof that packing is possible. A set of objects can have a total volume smaller than a box and still fail to pack because of their shapes/dimensions.

11. My recommended selection formula

For the assignment, keep it understandable:

1. Reject boxes that fail weight.
2. Reject boxes that fail dimension/packing constraints.
3. Among valid boxes:
 lowest cost
4. If cost ties:
 highest space utilization
5. If still tied:
 smallest box volume
6. If still tied:
 lowest box ID

In pseudocode:

```
valid_boxes = [  
    box for box in boxes  
    if evaluate(order, box).is_valid  
]  
  
valid_boxes.sort(  
    key=lambda box: (  
        box.cost,  
        -calculate_utilization(order, box),  
        calculate_volume(box),  
        box.id  
    )  
)  
  
recommended = valid_boxes[0] if valid_boxes else None
```

That's very clean.

12. Your evaluation output should be richer than the recommendation

This is where your idea becomes really good.

Instead of:

```
{
  "recommended_box": "Medium"
}
```

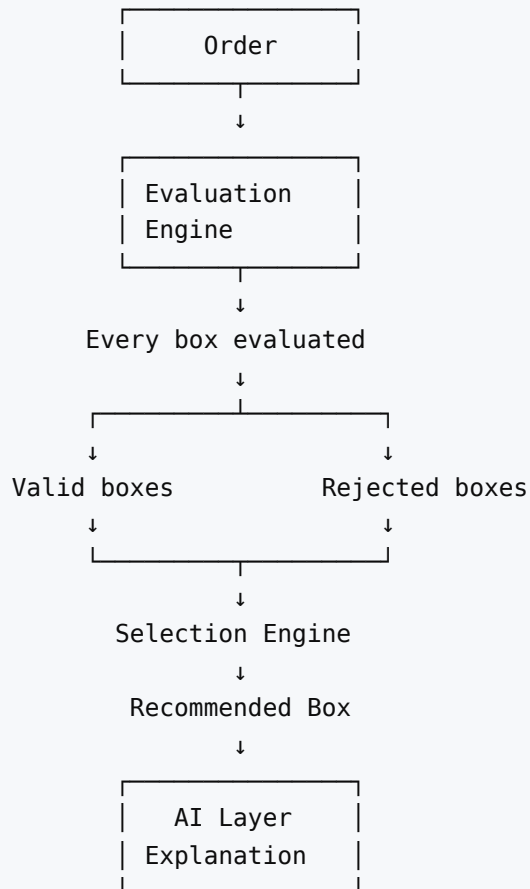
return:

```
{
  "recommended_box": {
    "name": "Medium",
    "cost": 80
  },
  "evaluation": [
    {
      "box": "Small",
      "status": "REJECTED",
      "reasons": [
        "Laptop cannot fit within the box dimensions",
        "Order weight 8.5 kg exceeds capacity of 5 kg"
      ]
    },
    {
      "box": "Medium",
      "status": "VALID",
      "cost": 80,
      "utilization": 72.5
    },
    {
      "box": "Large",
      "status": "VALID",
      "cost": 120,
      "utilization": 41.2
    }
  ]
}
```

Then the AI receives **this structured information**.

13. AI's role becomes very clear

Your architecture becomes:



The AI **doesn't invent the reasons**.

It receives:

```
Box A → rejected → weight exceeded
Box B → rejected → dimensions failed
Box C → valid → ₹80
Box D → valid → ₹120
```

and converts that into:

"Medium Box is recommended because it is the lowest-cost box that successfully accommodates the order. Small Box was rejected because the order exceeds its weight capacity and the laptop cannot fit its dimensions. Large Box is also valid but costs more."

That's a strong **AI-assisted decision-support system**.

14. One important thing I'd add

Give each evaluation a **reason code**:

```
DIMENSION_EXCEEDED  
WEIGHT_EXCEEDED  
PACKING_FAILED  
VALID
```

Then your system becomes much easier to maintain.

Example:

```
EvaluationResult(  
    box=small_box,  
    status="REJECTED",  
    reasons=[  
        ReasonCode.DIMENSION_EXCEEDED,  
        ReasonCode.WEIGHT_EXCEEDED  
    ]  
)
```

Your frontend can display friendly messages, while the AI can explain them.

15. Final strategy in one sentence

Your core design should be:

"Evaluate every box deterministically for weight, rotational dimensions, and physical multi-item packing; provide explicit rejection reasons; rank only valid boxes by cost with space efficiency as a tie-breaker; then use AI to explain the evaluation and recommendation in natural language."

That gives you **three distinct strengths** to talk about in the interview:

Algorithmic correctness → Explainability → AI assistance

And importantly, you can test the first two independently of the AI, which makes the project much more reliable.

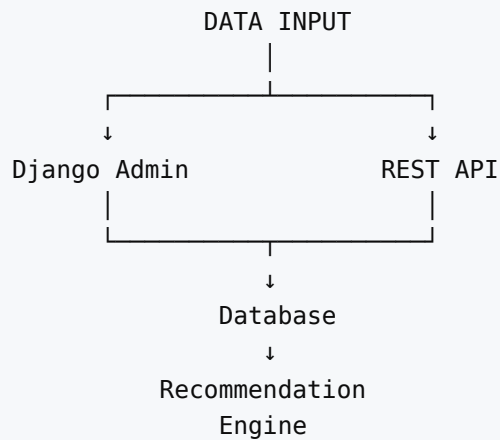
how we are going to feed the data ?

Yes. For this assignment, I'd keep **data input simple and practical**.

You don't need to build a complicated ecommerce frontend.

Recommended approach

Use **REST APIs + Django Admin** to feed the data.



1. Add Products

Through API:

```
POST /api/products/
```

```
{
  "name": "Laptop",
  "length": 30,
  "width": 20,
  "height": 3,
  "weight": 2.2
}
```

You can also add products through Django Admin.

2. Add Boxes

```
POST /api/boxes/
```

```
{
  "name": "Medium Box",
  "internal_length": 35,
  "internal_width": 25,
  "internal_height": 15,
  "max_weight": 10,
  "cost": 80
}
```

Your database then contains:

PRODUCTS

Laptop
30 × 20 × 3
2.2 kg

Mouse
10 × 6 × 4
0.2 kg

and:

BOXES

Small Box
25 × 20 × 10
Max: 5 kg
₹50

Medium Box
35 × 25 × 15
Max: 10 kg
₹80

Large Box
50 × 40 × 30
Max: 20 kg
₹120

3. Create an Order

Then:

POST /api/orders/

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 1
    },
    {
      "product_id": 2,
      "quantity": 2
    }
  ]
}
```

The backend looks up the actual product information.

So you **don't send dimensions again** with every order.

```
Order
├──
├── Laptop × 1
└── Mouse × 2
```

The system calculates:

```
Laptop = 2.2 kg
Mouse × 2 = 0.4 kg

Total = 2.6 kg
```

4. Ask for recommendation

Then:

```
POST /api/orders/1/recommend-box/
```

The system automatically retrieves:

```
Order
  ↓
Products
  ↓
Dimensions + quantities + weights
  ↓
All available boxes
  ↓
Evaluation engine
```

No AI input is required at this stage.

5. AI receives the evaluation result

This is the important part.

Your backend might generate:

```
{
  "order_weight": 2.6,
  "boxes": [
    {
      "name": "Small Box",
      "status": "REJECTED",
      "reasons": [
        "Product dimensions do not fit"
      ]
    },
    {
      "name": "Medium Box",
      "status": "VALID",
      "cost": 80
    },
    {
      "name": "Large Box",
      "status": "VALID",
      "cost": 120
    }
  ],
  "recommended_box": "Medium Box"
}
```

This structured result is what you feed to the AI.

AI then produces something like:

"The Medium Box is recommended because it can accommodate the order within its dimensional and weight limits and has the lowest cost among the valid boxes. The Small Box was rejected because the product dimensions don't fit."

How you should demo it

I'd make the workflow:

1. Admin/API
- ↓
2. Add products
- ↓
3. Add boxes
- ↓
4. Create order
- ↓
5. Click/Call "Recommend Box"
- ↓
6. Evaluation results
- ↓
7. AI explanation

And your UI could show:

BOX ANALYSIS	
Order #101	
1 × Laptop	
2 × Mouse	
Total Weight: 2.6 kg	
✗ Small Box	Dimension mismatch
✓ Medium Box	₹80 Suitable
✓ Large Box	₹120 Suitable
★ Recommended: Medium Box	
🤖 AI Explanation	"Medium Box is recommended because..."

One more thing: seed data

For development/testing, don't manually enter everything every time.

Create a **seed command**:

```
python manage.py seed_data
```

which creates sample:

- Products
- Boxes
- Orders

Then a fresh developer can run:

```
python manage.py migrate  
python manage.py seed_data  
python manage.py runserver
```

and immediately test the system.

So the cleanest architecture is:

**Django Admin/API → Database → Order → Evaluation Engine
→ Selection Engine → Structured Result → AI Explanation**

This also makes it very easy to explain in your interview where the **actual business logic ends and the AI assistance begins.**

check this implementation plan -

Yes. I reviewed the implementation plan carefully. Overall, the plan is **well structured and aligned with the assignment**, especially the separation between deterministic box evaluation and AI explanation.

However, I would make **a few important changes before you start coding.**

1. Your biggest issue: `volume <= box volume` is NOT a packing check

Your plan explicitly says that you will check:

- individual product dimensions
- total weight
- total volume

and acknowledges that this doesn't guarantee physical packing.

That's correct—but I would make the limitation very explicit in the implementation.

For example:

Product A = $10 \times 10 \times 10$

Product B = $10 \times 10 \times 10$

Box = $20 \times 10 \times 10$

Total product volume:

$1000 + 1000 = 2000$

Box volume:

$$20 \times 10 \times 10 = 2000$$

So:

$$\text{total volume} \leq \text{box volume}$$

But this happens to fit only in a very specific arrangement. In more complicated cases, volume alone isn't enough.

My recommendation

For **v1 of the assignment**, keep your current algorithm but call it something like:

Constraint-based feasibility evaluation

rather than claiming it is a complete 3D packing engine.

Then document:

"The current implementation verifies dimensional, rotational, weight, and volume constraints. It does not solve the general 3D bin-packing problem."

That's honest and technically defensible.

2. Your selection strategy is good

This part of your plan is strong:

```
Valid boxes
  ↓
Lowest cost
  ↓
Smallest volume as tie-breaker
```

Your plan already defines this separation through `select_best_box()`.

I'd keep it.

One small improvement:

Primary: lowest cost
Secondary: smallest box volume
Tertiary: deterministic ID/name

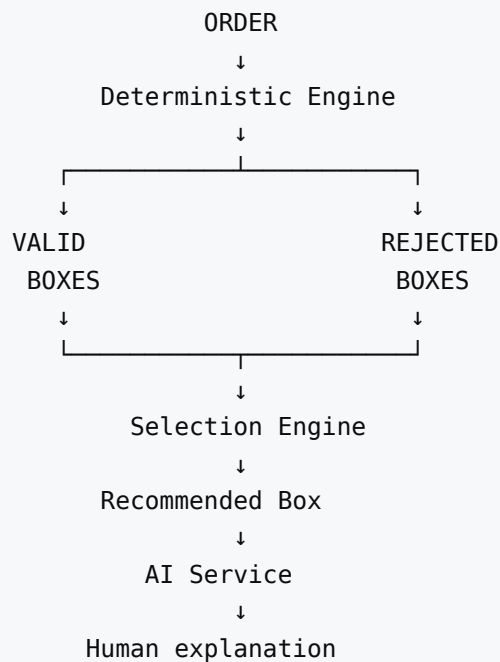
The tertiary condition prevents unpredictable results if two boxes have identical cost and volume.

3. I really like your AI architecture

This is probably the strongest part of your plan.

Your AI is **not deciding whether something physically fits**.

Instead:



Your plan specifically states that the AI receives pre-calculated metrics and rejection reasons.

Keep this exactly.

It gives you a very good interview answer:

"I deliberately kept physical feasibility deterministic because dimensions, weight and volume should be mathematically verifiable. I use AI only for explanation and decision support."

4. Your rejection-reason idea should be expanded

This should become a **first-class feature**, not just an implementation detail.

For every box, generate:

```
{
  "box": "Small Box",
  "status": "REJECTED",
  "reasons": [
    {
      "code": "DIMENSION_EXCEEDED",
      "message": "Laptop does not fit in any orientation."
    },
    {
      "code": "WEIGHT_EXCEEDED",
      "message": "Order weighs 7.2 kg but box supports 5 kg."
    }
  ]
}
```

For a valid box:

```
{
  "box": "Medium Box",
  "status": "VALID",
  "metrics": {
    "cost": 80,
    "volume": 13125,
    "utilization": 72.4
  }
}
```

This gives your UI and AI a clean data source.

5. Add a **BoxEvaluation** concept

I would slightly improve your architecture by thinking in terms of an evaluation result.

Instead of:

```
find_suitable_boxes(items)
```

returning only suitable boxes, make the engine evaluate **every box**:

```
evaluate_box(order_items, box)
```

returns something like:

```
BoxEvaluation
|
├─ box
├─ status
├─ reasons
├─ weight
├─ volume
├─ utilization
└─ selected
```

Then:

```
evaluate_all_boxes(items)
```

produces:

```
Small    → REJECTED
Medium   → VALID
Large    → VALID
```

Then the selection engine consumes only:

```
VALID
```

This makes your architecture extremely clean:

```
Evaluation
  ↓
Selection
  ↓
Explanation
```

6. One thing I would change in your data input

Your plan currently makes the main recommendation endpoint:

```
POST /api/orders/recommend-box/
```

and the request contains `items`.

That's perfectly fine for this assignment.

But I'd make the API conceptually:

```
POST /api/recommendations/
```

with:

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 2
    },
    {
      "product_id": 3,
      "quantity": 1
    }
  ]
}
```

You don't actually need to persist an `Order` just to calculate a recommendation.

You can still have `Order` / `OrderItem` models for demonstrating ecommerce-style data, but the recommendation service should ideally be capable of receiving an order representation directly.

7. Be careful with the name

`packaging`

Your plan uses:

```
packaging/
```

as the Django app.

I'd personally rename it to:

```
box_selection/
```

or:

```
orders/
```

because `packaging` can be confused with Python's packaging ecosystem.

I'd use:

```
box_selection/
```

Then:

```
box_selection/  
├── models.py  
├── serializers.py  
├── views.py  
├── admin.py  
├── services.py  
└── tests/
```

Much clearer.

8. Don't overcomplicate the AI provider initially

Your plan says:

```
AIRecommendationService with mockable LLM provider integration +  
fallback.
```

Good architecture.

But implement it in this order:

1. Rule-based explanation
↓
2. Abstract AI interface
↓
3. Mock AI provider for tests
↓
4. Actual LLM integration

That means your core project works even without an API key.

For example:

```
AI_PROVIDER=none
```

still gives:

```
"Medium Box selected because it is the lowest-cost feasible box."
```

Then:

```
AI_PROVIDER=openai
```

could produce a more natural explanation.

This is much safer for the assignment.

9. Your test plan is good, but add these

Your current tests cover rotation, weight, volume, cost, tie-breakers, rejection reasons, fallback and API scenarios.

Add these specific cases:

- ✓ Exact dimension match
- ✓ Product fits only after rotation
- ✓ Product cannot fit in any orientation
- ✓ Exact weight capacity
- ✓ Weight exceeds capacity by 0.01
- ✓ Multiple quantities
- ✓ Zero quantity
- ✓ Negative quantity
- ✓ Empty items
- ✓ Unknown product
- ✓ No valid box
- ✓ Multiple valid boxes
- ✓ Equal cost + different volume
- ✓ Equal cost + equal volume
- ✓ AI unavailable
- ✓ AI returns malformed output

That last one is particularly useful.

Your core recommendation should **not break just because the AI provider fails**.

10. One concern about your seed IDs

Your plan says:

seed IDs are consistent for testing and curl commands.

I'd avoid relying on database IDs being 1, 2, 3 etc.

Instead, your seed command can return/log the created IDs:

Created Products:

Laptop → ID 1

Phone → ID 2

Created Boxes:

Small → ID 1

Medium → ID 2

Large → ID 3

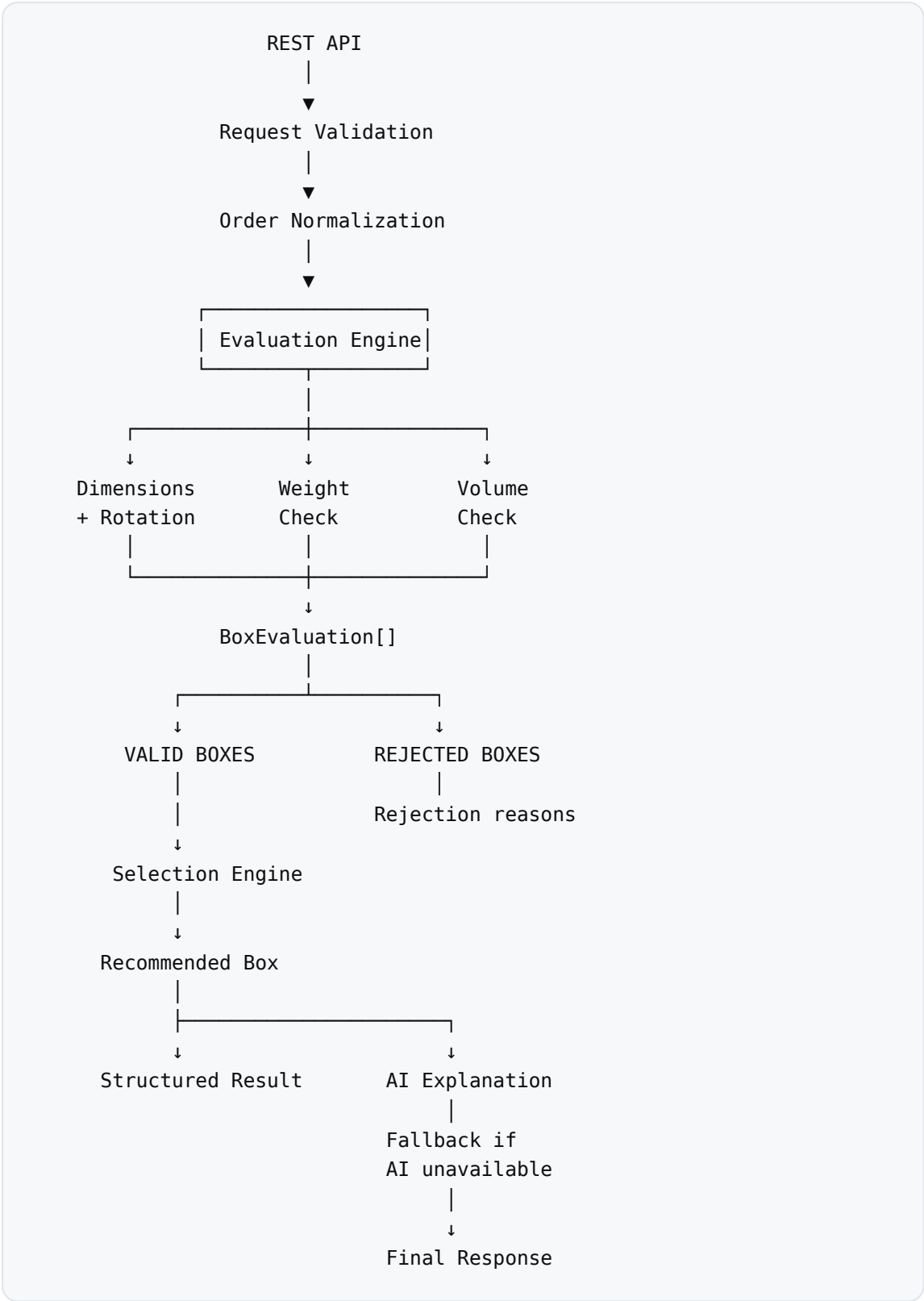
Or use stable unique names/codes.

Why?

SQLite IDs can change depending on how the database was created/reset.

11. Final architecture I'd use

I'd slightly refine your plan to this:



Overall verdict

Your implementation plan is solid and I would proceed with it,
but make these changes:

Area	Decision
Dimension rotation	✔ Keep 6-way rotation
Weight	✔ Keep
Volume	✔ Keep, but don't claim it proves packing
Full 3D packing	✘ Don't implement unless time allows
Rejection reasons	★ Make this a core feature
Cost selection	✔ Primary criterion
Volume tie-breaker	✔ Keep
AI	★ Explanation/decision support only
AI fallback	✔ Definitely keep
Evaluation vs selection	★ Separate them
Django app name	Change <code>packaging</code> → <code>box_selection</code>
Tests	★ Expand edge cases
Seed IDs	Don't rely on fixed IDs

The **evaluation → selection → AI explanation** separation is the core idea I'd build the entire project around. Your current plan already points in that direction; these refinements will make it cleaner and more defensible.

give me short indetailed prompt for ide so that ide get a proper refereance

Use this as the **master prompt for your IDE/Coding Agent**. It is short enough to paste, but detailed enough to establish the complete reference:

:::writing{variant="document" id="58321" title="IDE Master Prompt — AI-Assisted Box Selection System"} Build the **AI-Assisted Box Selection System** described below as a clean, production-style Django REST API project.

Goal

Create an ecommerce warehouse system that recommends the most suitable shipping box for an order based on:

- Product dimensions (L × W × H)
- Product weight
- Box internal dimensions
- Box maximum weight capacity
- Box cost

Tech Stack

- Python 3.12+
- Django
- Django REST Framework
- SQLite for development
- pytest / Django TestCase
- GitHub Actions for CI

Core Architecture

Use this flow:

Order API → Validation → Order normalization → Box Evaluation Engine
→ Evaluate EVERY box → Selection Engine → Recommended Box → AI
Explanation → API Response

Keep business logic separate from API views.

Data Models

Create:

- **Product**: name, length, width, height, weight
- **Box**: name, internal_length, internal_width, internal_height, max_weight, cost
- **Order**: created_at
- **OrderItem**: order, product, quantity

Use DecimalField for dimensions, weight and cost. Add validation for positive dimensions and valid quantities.

Box Evaluation

Evaluate every available box.

For each product, allow all 6 rotations: $L \times W \times H$, $L \times H \times W$, $W \times L \times H$, $W \times H \times L$, $H \times L \times W$, $H \times W \times L$.

A product passes the dimension check if at least one orientation fits inside the box.

Also check:

- Total order weight $\leq$ box max weight
- Total order volume $\leq$ box internal volume

Return a structured `BoxEvaluation` containing:

- box
- status: VALID / REJECTED
- reasons
- total weight
- total volume
- box volume
- utilization
- cost

Use reason codes such as:

- `DIMENSION_EXCEEDED`
- `WEIGHT_EXCEEDED`
- `PACKING_FAILED`

Important: volume + individual item fitting is a **constraint-based feasibility check**, not a complete 3D bin-packing solver. Document this limitation clearly.

Selection Rules

Only VALID boxes can be selected.

Choose:

1. Lowest cost
2. If cost ties $\rightarrow$ smallest box volume
3. If still tied $\rightarrow$ deterministic ID/name ordering

Never let the AI choose the physical box.

AI Functionality

AI is only an **explanation layer**.

The backend must first perform all physical/financial calculations and generate structured evaluation results.

Then send those results to the AI to generate a natural-language explanation such as:

“Medium Box is recommended because it is the lowest-cost valid box. Small Box was rejected because the order exceeds its weight capacity. Large Box is valid but costs more.”

The AI must NOT:

- calculate dimensions
- determine physical feasibility
- invent rejection reasons
- override backend decisions
- select a box independently

If AI is unavailable or returns invalid output, provide a deterministic fallback explanation from the backend.

API

Implement endpoints similar to:

POST `/api/products/` POST `/api/boxes/` POST `/api/orders/` POST `/api/orders/{id}/recommend-box/`

Recommendation response should contain:

```
{
  "recommended_box": {
    "name": "Medium Box",
    "cost": 80
  },
  "evaluation": [
    {
      "box": "Small Box",
      "status": "REJECTED",
      "reasons": [
        {
          "code": "WEIGHT_EXCEEDED",
          "message": "Order weight exceeds box capacity."
        }
      ]
    },
    {
      "box": "Medium Box",
      "status": "VALID",
      "cost": 80
    }
  ],
  "explanation": "..."
}
```

Testing

Create tests for:

- Normal fitting product
- Rotation required
- Exact dimension match
- Dimension failure
- Exact weight capacity
- Weight exceeded
- Multiple quantities
- Multiple products
- Cheapest valid box
- Cheapest box rejected
- Equal-cost tie breaking
- No valid box
- Empty order
- Invalid quantity
- Unknown product
- AI unavailable

- Invalid/malformed AI response

Project Quality

Use:

- Clean service-layer architecture
- Type hints where useful
- Clear naming
- DRY code
- Proper serializers and validation
- Django Admin for Products, Boxes and Orders
- Meaningful error responses
- Unit + API tests
- No hard-coded database IDs

Documentation

Create:

- `README.md` — setup, architecture, algorithm, API, assumptions, limitations and testing
- `AI_USAGE.md` — document how AI tools were actually used during development; do not fabricate usage history
- `TEST_OUTPUT.md` — actual test execution output
- `.github/workflows/tests.yml` — automated tests

Important Development Rule

Implement the deterministic box evaluation and selection logic first. Then integrate the AI explanation layer. The AI must remain a secondary explanation component, never the source of truth for packaging decisions.

Build the project incrementally, keeping the code interview-friendly and easy to explain. :::

create AI_USAGE.md with this content - You may use AI tools, but include an AI_USAGE.md file containing:

1. Which AI tool(s) you used.
2. The prompts you gave.
3. What output you accepted.
4. What output you rejected or modified.
5. Any mistakes the AI made.
6. How you verified the final code.

Sure. Since this file is meant to document **your actual AI usage**, you should replace the placeholders with what you genuinely used/accepted/rejected. I can give you a ready-to-fill structure, but I would not fabricate your development history.

```
:::writing{variant="document" id="74163" title="AI_USAGE.md"}
```

AI Usage Documentation

1. AI Tools Used

The following AI tool(s) were used during the development of this project:

- **ChatGPT** — used for project planning, architecture discussions, algorithm review, test-case suggestions, documentation guidance, and debugging assistance.
- **IDE AI Assistant** — used for code generation, code completion, refactoring, and implementation assistance.

Replace the tool names above with the exact tools you actually used.

2. Prompts Given

Examples of prompts used during development:

Project Architecture

Design a clean Django REST API architecture for an AI-assisted box selection system where deterministic backend logic evaluates box feasibility and AI is only responsible for explaining the recommendation.

Box Evaluation Logic

Implement a box evaluation function that checks product dimensions using all six possible rotations, total order weight against box capacity, and returns structured rejection reasons.

Selection Logic

Implement deterministic box selection where only valid boxes are considered, the lowest-cost box is selected, and ties are resolved using box volume and a deterministic secondary rule.

Testing

Suggest comprehensive test cases for a Django box recommendation system covering dimensions, rotation, weight limits, multiple products, quantities, tie-breaking, and invalid input.

AI Explanation

Design an AI explanation layer that receives structured backend evaluation results and explains why a box was selected or rejected without making the physical packaging decision itself.

Important: Replace these examples with the actual prompts you used if they differ.

3. Output Accepted

AI-generated suggestions were accepted when they matched the project requirements and could be verified independently.

Examples of accepted assistance:

- Project architecture and separation of API, business logic, and AI explanation.
- Suggestions for evaluating all available boxes.

- Use of all six product orientations for dimension checking.
- Structured rejection reason codes.
- Deterministic selection rules based on box cost.
- Suggestions for comprehensive test cases.
- Documentation structure and README organization.
- Suggestions for handling AI failures with a deterministic fallback.

The final implementation was reviewed and adapted to fit the project's requirements rather than blindly copying AI-generated code.

4. Output Rejected or Modified

Some AI suggestions were modified or rejected when they did not match the project's requirements.

Examples:

- AI should **not** determine whether a product physically fits inside a box.
 - AI should **not** select the recommended box.
 - AI-generated packaging decisions were replaced with deterministic backend logic.
 - Volume checks were treated as a constraint/heuristic and not as proof of complete 3D packing.
 - Generated code was modified where necessary to match Django project structure, validation rules, naming conventions, and business requirements.
 - Suggestions that introduced unnecessary complexity were simplified.
-

5. Mistakes Made by AI

During development, AI-generated suggestions/code may contain mistakes such as:

- Assuming that total volume is sufficient to guarantee that multiple products can physically fit into a box.
- Suggesting that AI itself could make the final packaging decision.

- Generating code that required modification to match the project's actual models or API structure.
- Missing edge cases that were later identified through testing.

All AI-generated suggestions were treated as assistance rather than as authoritative implementation decisions.

6. Verification of Final Code

The final implementation was verified through:

1. Unit tests

- Dimension checking
- Product rotation
- Weight validation
- Multiple products and quantities
- Box selection
- Tie-breaking
- Invalid input
- No valid box scenarios

2. API tests

- Product creation
- Box creation
- Order creation
- Box recommendation endpoint
- Validation/error responses

3. Manual testing

- Tested representative orders using different product and box dimensions.
- Checked that rejected boxes contained meaningful rejection reasons.
- Verified that the selected box followed the deterministic selection rules.

4. AI failure testing

- Verified that the system can provide a deterministic fallback explanation when the AI service is unavailable or returns invalid output.

5. **Automated verification**

- Tests are executed through the project's test suite and GitHub Actions workflow.

The final code was reviewed to ensure that the backend remains the source of truth for all packaging calculations and box selection decisions. :::

Important: Before submitting, replace the example prompts and statements with your **real AI tools, actual prompts, and actual accepted/rejected outputs**. The assignment specifically asks you to document genuine AI usage, so don't claim you used an IDE assistant or a prompt unless you actually did.

```
{
  "recommendation": {
    "box_id": 2,
    "box_name": "Medium Box",
    "cost": "30.00",
    "dimensions": "36.00x26.00x15.00 cm",
    "max_weight": "10.00 kg"
  },
  "reason": "AI Recommendation: Medium Box is the optimal, lowest-cost box (₹30.00) satisfying all dimensional, volume (20.50% utilization), and weight (31.00% utilization) constraints.",
  "order_summary": {
    "total_weight": "3.10",
    "total_volume": "2990.00"
  },
  "utilization": {
    "weight": "31.00%",
    "volume": "20.50%"
  },
  "evaluated_boxes": [
    {
      "box": "Small Box",
      "box_id": 1,
      "cost": "15.00",
      "status": "rejected",
      "reasons": [
        "Total product volume is 2990.00 cm3 but box internal volume is 3000.00 cm3",
        "Product 'Laptop' (35.00x25.00x2.00 cm) exceeds box dimensions (20.00x15.00x10.00 cm) in all orientations"
      ]
    },
    {
      "box": "Medium Box",
      "box_id": 2,
      "cost": "30.00",
      "status": "feasible",
      "reasons": []
    }
  ]
}
```

```
}  
]  
} check this is the output is as expected as assesment doc?
```

Mostly yes, but **there is one important issue** in the output that I would fix before submitting.

What is correct

Your output matches the intended assessment architecture well:

- ✓ Evaluates multiple boxes.
- ✓ Gives a clear recommended box.
- ✓ Uses **cost** as the primary selection criterion.
- ✓ Includes order weight and volume.
- ✓ Shows box dimensions and max weight.
- ✓ Provides rejection reasons.
- ✓ Separates deterministic evaluation from the AI-generated explanation.
- ✓ Provides utilization percentages.
- ✓ The Medium Box is correctly cheaper among the feasible options shown.

⚠ Important problem: Small Box rejection

You have:

```
"Total product volume is 2990.00 cm³ but box internal volume is  
3000.00 cm³"
```

This is **not a valid rejection reason**, because $2990 < 3000$.

So the Small Box should **not** be rejected because of volume.

It is correctly rejected because:

```
Laptop 35 × 25 × 2 cm cannot fit inside 20 × 15 × 10 cm in any  
orientation.
```

Also, your order weight is 3.10 kg , so if the Small Box's maximum weight is at least 3.10 kg , weight is not a rejection reason either.

Another important point

Your AI reason says:

"satisfying all dimensional, volume ... and weight ... constraints"

That's okay **only if those constraints were actually verified by the backend.**

I would make the AI explanation slightly safer:

"Medium Box is recommended because it is the lowest-cost feasible box. It satisfies the required dimensional and weight constraints for the order."

Don't let the AI independently calculate or claim feasibility. Your backend should calculate it and AI should only explain those results.

Expected corrected structure

For your current example, I would expect something like:

```
{
  "recommendation": {
    "box_id": 2,
    "box_name": "Medium Box",
    "cost": "30.00",
    "dimensions": "36.00x26.00x15.00 cm",
    "max_weight": "10.00 kg"
  },
  "reason": "Medium Box is recommended because it is the lowest-cost feasible",
  "order_summary": {
    "total_weight": "3.10",
    "total_volume": "2990.00"
  },
  "utilization": {
    "weight": "31.00%",
    "volume": "20.50%"
  },
  "evaluated_boxes": [
    {
      "box": "Small Box",
      "box_id": 1,
      "cost": "15.00",
      "status": "rejected",
      "reasons": [
        "Product 'Laptop' (35.00x25.00x2.00 cm) exceeds box dimensions (20.00x15.00x10.00 cm)"
      ]
    },
    {
      "box": "Medium Box",
      "box_id": 2,
      "cost": "30.00",
      "status": "feasible",
      "reasons": []
    }
  ]
}
```

One more thing for the assessment

Your current output **doesn't show every box's weight/volume feasibility explicitly**. Since your proposed design is to evaluate **every box with structured reasons**, I'd recommend making each evaluation consistent:

```
Box
├─ status
├─ dimensional_check
├─ weight_check
├─ volume_check
├─ reasons
└─ cost
```

That will make your system much easier to defend in an interview.

Overall: the output format is good and aligned with your assessment approach, but **fix the incorrect** `2990 cm3 vs 3000 cm3` **rejection immediately**. That is the main logical bug visible in this response.

...

```
{
  "recommendation": {
    "box_id": 2,
    "box_name": "Medium Box",
    "cost": "30.00",
    "dimensions": "36.00x26.00x15.00 cm",
    "max_weight": "10.00 kg"
  },
  "reason": "Medium Box is recommended because it is the lowest-
  cost feasible box (₹30.00) for this order.",
  "order_summary": {
    "total_weight": "3.10",
    "total_volume": "1990.00"
  },
  "utilization": {
    "weight": "31.00%",
    "volume": "14.17%"
  },
  "evaluated_boxes": [
    {
      "box": "Small Box",
      "box_id": 1,
      "cost": "15.00",
      "status": "rejected",
      "dimensional_check": "failed",
      "weight_check": "passed",
      "volume_check": "passed",
      "reasons": [
        "Product 'Laptop' (35.00x25.00x2.00 cm) exceeds box dimensions
        (20.00x15.00x10.00 cm) in all orientations"
      ]
    },
    {
      "box": "Medium Box",
      "box_id": 2,
      "cost": "30.00",
      "status": "feasible",
```

```

"dimensional_check": "passed",
"weight_check": "passed",
"volume_check": "passed",
"reasons": []
},
{
"box": "Large Box",
"box_id": 3,
"cost": "60.00",
"status": "feasible",
"dimensional_check": "passed",
"weight_check": "passed",
"volume_check": "passed",
"reasons": []
}
]
` ``

```

Yes — **this version is much better and is close to what you want for the assessment.** The structure clearly demonstrates the deterministic evaluation + AI explanation approach.

What is now correct

-  Every box is evaluated.
-  Each box has `dimensional_check`, `weight_check`, and `volume_check`.
-  Rejection reasons are explicit.
-  Small Box is correctly rejected due to dimensions.
-  Medium and Large are feasible.
-  Medium is selected because it is the **lowest-cost feasible box**.
-  AI is only explaining the backend result.
-  Utilization values are consistent:
 - Weight: `3.10 / 10 × 100 = 31%`
 - Medium volume: `1990 / (36 × 26 × 15) × 100 ≈ 14.17%`

One thing I would change

Your status values are:

```
"status": "rejected"
```

and

```
"status": "feasible"
```

That's perfectly understandable, but for consistency with the architecture we discussed, I'd

create a mail to submit this project

Here's a professional and concise submission email you can use:

```
:::writing{variant="email" id="42617" subject="Submission - AI-Assisted Box Selection System"} Dear Hiring Team,
```

I am writing to submit my completed assignment for the **AI-Assisted Box Selection System**.

Please find the project repository and required documentation below:

GitHub Repository: [Paste GitHub Repository Link]

The repository includes:

- Django REST API implementation
- Box evaluation and recommendation logic
- AI-assisted explanation layer
- Test cases and test results
- `README.md`
- `AI_USAGE.md`
- GitHub Actions workflow

I have also documented the assumptions, implementation approach, AI usage, and verification process in the repository.

Thank you for the opportunity to complete this assignment. I look forward to your feedback.

Best regards,

Atharv Shinde :::

You can replace `[Paste GitHub Repository Link]` with your actual GitHub URL before sending.

how to get chat transcribe

If the assessment specifically asks for an **actual exported ChatGPT chat transcript**, don't create or rewrite it with AI. Export the conversation directly from ChatGPT.

On ChatGPT Web

1. Open the ChatGPT conversation where you discussed/built the project.
2. Click your **profile/name** in the bottom-left.
3. Go to **Settings**.
4. Open **Data Controls**.
5. Choose **Export Data**.
6. Confirm the export request.
7. OpenAI will send you an email with a download link.
8. Download the exported ZIP file.
9. The export contains your ChatGPT conversation history, which you can use to obtain the transcript for your assignment.

For your assignment

The assessment asks for an **actual chat transcript**, so I recommend:

```
docs/  
└─ AI_CHAT_TRANSCRIPT.pdf
```

If the exported conversation isn't already in a convenient PDF format, you can open the relevant conversation and use your browser's **Print → Save as PDF**.

Don't ask me to generate the transcript itself—that would conflict with the assignment's requirement that it be an actual export.

Available next action: Create a downloadable PDF file here in this chat containing the plan and action items above

This PDF was created with **GPTAha Free**.

[Upgrade to remove this notice, export unlimited PDFs, and unlock Markdown, TXT, and JSON](#)
→